

Mentor: Darsi Gangothri

Session 5 - Chatbot, Streaming, Pydantic, Async + Neural Networks

Date: Saturday, 28 Jun 2026

Module: 2.1 (Part 2) - Advanced Python for AI + Neural Network Foundations

Course: LLM Engineering Masterclass

What We Covered

Session Overview

This session completes Module 2.1 by covering all remaining Python tools needed for AI engineering, plus introducing the foundational concepts of how AI models actually work internally (neural networks and transformers).

By the end of this session, students will understand:

- How chatbots maintain conversation history (memory)
- How streaming makes AI responses feel real-time
- How Pydantic validates AI outputs for production safety
- How async/await runs multiple AI calls in parallel
- How to read and write files (including JSON configs)
- What virtual environments are and why they matter
- How a neural network learns (neurons, layers, training loop)
- What self-attention is (the key idea behind transformers)

Topic 1: Chatbot with Memory

What is it?

A chatbot is an AI program that has a conversation with you. But by default, AI models have NO memory. Each time you call the API, it's a fresh start -- the model doesn't know what you said before.

The Problem

If you ask: 1. "What is the capital of France?" → AI says "Paris" 2. "What language do they speak there?" → AI says "I don't know what 'there' means"

The second call doesn't know about the first. Each API call is independent.

The Solution: Conversation History

We maintain a **list** that grows with every exchange. Every time we call the AI, we send the ENTIRE list (all previous messages). The model reads everything and responds in context.

The model doesn't "remember" -- we remind it every time by sending the full history.

How It Works (Gemini API)

```
history = []
system_instruction = "You are a friendly AI assistant."

# User says something
history.append({"role": "user", "parts": [{"text": "Capital of France?"}]})

# Send full history to AI
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=history,
    config={"system_instruction": system_instruction}
)

# Add AI response to history
history.append({"role": "model", "parts": [{"text": response.text}]})
```

Key Points

- **role: "user"** = what the human said
- **role: "model"** = what the AI said
- The list grows by 2 every turn (user message + model response)
- Turn 1: 2 messages. Turn 5: 10 messages. Turn 50: 100 messages.
- This is how ChatGPT, Gemini, and every chatbot works

Limitation

The history can get very long. Models have a maximum context length. Eventually you need strategies like summarization or sliding windows -- but that's for later sessions.

Topic 2: Streaming

What is it?

Streaming means receiving the AI's response **word by word** as it's being generated, instead of waiting for the entire response to finish.

Without Streaming (Normal)

1. You send a question
2. You wait 5-10 seconds staring at nothing
3. The FULL answer appears all at once

With Streaming

1. You send a question
2. Words start appearing immediately, one by one
3. Feels like someone is typing live

Why It Matters

- Better user experience (no awkward silence)
- Every AI product uses it (ChatGPT, Gemini, Claude)
- Users perceive faster responses even if total time is the same

How It Works (Gemini API)

```
def ask_ai_stream(prompt):
    response = client.models.generate_content_stream(
        model="gemini-2.5-flash",
        contents=prompt,
        config={"system_instruction": "You are a helpful assistant."}
    )

    full_response = ""
    for chunk in response:
        word = chunk.text
        print(word, end="", flush=True)
        full_response += word

    print()
    return full_response
```

Key Differences from Normal Call

Normal (<code>generate_content</code>)	Streaming (<code>generate_content_stream</code>)
Returns one response object	Returns an iterator of chunks
Wait for full answer	Get pieces immediately
<code>response.text</code> gives everything	Loop through chunks one by one

The Magic Line

`print(word, end="", flush=True)` -- prints each piece side by side (no newline), immediately (flush=True forces output without buffering).

Topic 3: Pydantic (Data Validation)

What is it?

Pydantic is a Python library that validates data. You define what data SHOULD look like (a "blueprint"), and Pydantic checks if actual data matches that blueprint.

The Problem with AI Outputs

AI returns plain text. It's unpredictable: - Sometimes proper JSON: `{"name": "John", "age": 25}` - Sometimes plain English: "John is 25 years old" - Sometimes wrong

format or missing fields - Sometimes garbage

If your app expects a number and gets "a lot" -- it crashes.

The Solution: Pydantic Models

```
from pydantic import BaseModel

class Person(BaseModel):
    name: str
    age: int
    city: str
```

This says: "A valid Person MUST have name (text), age (whole number), city (text)."

How Validation Works

```
# VALID -- accepted
person = Person(name="Priya", age=24, city="Mumbai")

# INVALID -- REJECTED with clear error
person = Person(name="Rahul", age="twenty-five", city="Delhi")
# Error: age must be an integer
```

Using Pydantic with AI

1. Ask AI to respond in JSON format
2. Parse the JSON string into a Python dict
3. Pass it to your Pydantic model
4. If valid → use the clean data
5. If invalid → handle the error gracefully

```
import json

response = ask_ai("Give me a person as JSON: {name, age, city}")

# Strip markdown formatting if AI wraps in ```json
clean = response.strip()
if clean.startswith("```"):
    clean = clean.split("\n", 1)[1]
    clean = clean.rstrip("```", 1)[0]

data = json.loads(clean)
person = Person(**data) # Validates!
```

Why This Matters in Production

- Medical apps: dosage MUST be a number
- Banking apps: transfer amount MUST be validated
- Any app where wrong data = real consequences
- Never pass unvalidated AI output to critical systems

Topic 4: Async/Await (Parallel Processing)

What is it?

Async (asynchronous) programming lets you run multiple tasks at the same time instead of one after another.

The Restaurant Analogy

Synchronous waiter: - Takes order from Table 1 → walks to kitchen → WAITS until food ready → delivers → THEN goes to Table 2 - 3 tables × 5 minutes each = 15 minutes

Asynchronous waiter: - Takes order from Table 1 → sends to kitchen → immediately goes to Table 2 → sends → goes to Table 3 - Kitchen makes all three in parallel - 3 tables = 5 minutes total

In Our World

Each "table" is an AI call. Without async, 3 AI calls taking 5 seconds each = 15 seconds. With async, all 3 run simultaneously = ~5 seconds.

Key Python Concepts

Keyword	Meaning
<code>async def</code>	This function can run alongside others
<code>await</code>	Pause here, but let other tasks work
<code>asyncio.gather()</code>	Run ALL tasks at once, collect all results

Code Example

```
import asyncio

async def ask_ai_async(prompt):
    return await asyncio.to_thread(ask_ai, prompt)

async def ask_all():
    tasks = [ask_ai_async(q) for q in questions]
    return await asyncio.gather(*tasks)

answers = await ask_all()
```

When to Use

- Multiple AI calls (summarize 10 documents)
- Translations (translate to 5 languages)
- Batch processing (grade 50 essays)
- Any time you're waiting on external services

Topic 5: File I/O + JSON

What is I/O?

I/O = Input/Output - **Input** (Read): Getting data FROM a file INTO Python - **Output** (Write): Sending data FROM Python TO a file

Why for AI?

- Store prompts in files (change behavior without code changes)
- Save conversation logs
- Load documents for processing
- Configuration files (model name, temperature, system prompts)

Basic File Operations

```
# WRITE a text file
with open("hello.txt", "w") as f:
    f.write("Hello from Python!")

# READ a text file
with open("hello.txt", "r") as f:
    content = f.read()
```

- `"w"` = write mode (creates or overwrites)
- `"r"` = read mode
- `with` = automatically closes the file when done

JSON (Structured Data)

JSON looks like Python dictionaries. It's the format AI APIs use.

```
import json

# Python dict → JSON file
config = {"model": "gemini-2.5-flash", "temperature": 0.7}
with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

# JSON file → Python dict
with open("config.json", "r") as f:
    config = json.load(f)
```

Practical Use

Change `config.json` → change AI behavior → no code changes needed. This is how production apps are configured.

Topic 6: Virtual Environments

What is it?

A virtual environment is an isolated space for a Python project. Each project gets its own packages -- no conflicts with other projects.

The Problem

- Project A needs `requests 2.28`
- Project B needs `requests 2.31`
- Both installed globally? They conflict. One breaks.

The Solution

Virtual environments = separate rooms. Each project has its own package versions.

Five Commands (All You Need)

Step	Command	What it does
1	<code>python -m venv myenv</code>	Create the environment
2	<code>source myenv/bin/activate</code>	Enter the environment
3	<code>pip install google-genai</code>	Install packages (only in this env)
4	<code>pip freeze</code>	See what's installed
5	<code>deactivate</code>	Exit the environment

Notes

- In Google Colab: already isolated (no need)
- On your laptop: ALWAYS use a virtual environment
- Other tools exist (Poetry, uv, conda) -- same concept, fancier

Topic 7: Neural Networks -- How AI Learns

What is a Neural Network?

A program that learns from examples instead of being told rules.

Traditional programming: You write rules → program follows them

Neural network: You show examples → it figures out the pattern

Example: Instead of writing rules to detect cats ("if pointy ears AND whiskers..."), show it 10,000 cat photos and 10,000 non-cat photos. It learns what makes a cat a cat.

What is a Neuron?

The smallest unit. It: 1. Takes **inputs** (numbers) 2. Multiplies each by a **weight** (how important is this input?) 3. **Sums** them up 4. Passes through an **activation** (threshold -- fire or stay quiet) 5. Outputs **one number**

Analogy: A hiring decision. Inputs: experience, education, interview. Weights: experience matters most (0.5), education (0.2), interview (0.3). If weighted sum > threshold → hire.

Layers and Networks

- **Layer:** Many neurons working in parallel (each finding different patterns)
- **Network:** Multiple layers stacked (output of layer 1 feeds layer 2)
- **Deep Learning:** Many layers = "deep"
 - Layer 1: Simple patterns (edges, colors)
 - Middle layers: Complex combinations (eyes, ears)
 - Last layer: Final decision (cat or not)

The Training Loop (How It Learns)

The model starts with **random weights** (random guesses). Then repeats:

1. **FORWARD PASS** -- Data goes in, prediction comes out
2. **LOSS** -- Compare prediction to correct answer. How wrong? (Loss = error)
3. **BACKWARD PASS** -- Trace backwards. Which weights caused the most error? (Backpropagation)
4. **UPDATE** -- Adjust those weights slightly

Repeat millions of times. Each time, loss gets smaller. Predictions get better.

Key Insight

The model IS the weights. When you download a 4GB model file, that's just billions of carefully tuned numbers. That's all an AI model is -- a file full of weights that were trained on data.

Topic 8: Transformer Teaser (Self-Attention)

What is a Transformer?

The specific type of neural network inside Gemini, ChatGPT, and all modern language AI. The key innovation: **self-attention**.

The Problem Without Attention

Sentence: "The cat sat on the mat because **it** was tired"

What does "it" refer to? You instantly know: the cat. But without attention, a model treats every word equally and can't figure out what "it" means.

Self-Attention

For every word, the model asks: "Which other words should I focus on to understand THIS word?"

- "it" → pays **strong attention** to "cat" (that's what it refers to)
- "it" → pays weak attention to "mat", "sat", etc. (less relevant)

Why It Matters

Without attention: model can't understand context, pronouns, or relationships between distant words.

With attention: model understands that "it" = "cat" -- which is why AI can have coherent conversations.

Next session covers the full architecture: multi-head attention, positional encoding, tokenizers, embeddings, and loading models with HuggingFace.

Session Summary

#	Topic	Key Takeaway
1	Chatbot Memory	Send full conversation history every call
2	Streaming	<code>generate_content_stream()</code> -- words appear live
3	Pydantic	Define data shape → validate → reject garbage
4	Async	<code>asyncio.gather()</code> -- parallel calls, 3x faster
5	File I/O	<code>json.dump()</code> / <code>json.load()</code> for configs
6	Virtual Envs	<code>python -m venv</code> -- isolated project packages

#	Topic	Key Takeaway
7	Neural Networks	Neurons → Layers → Network → Training Loop
8	Transformers	Self-attention = focus on relevant words

Homework

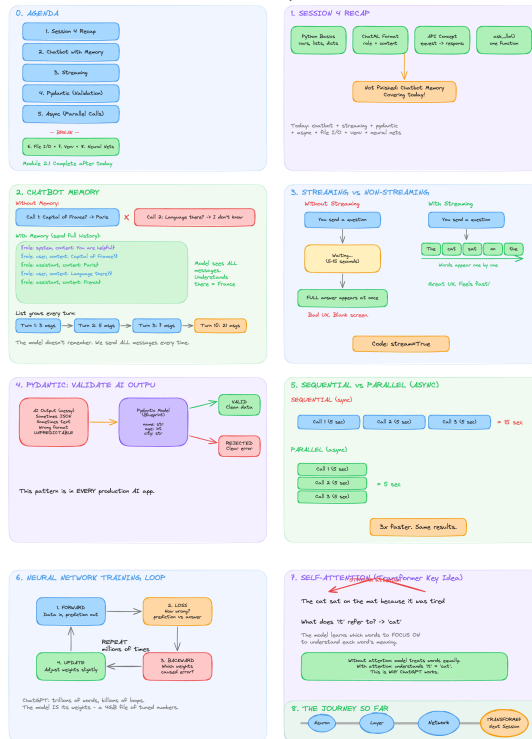
1. **Streaming Chatbot** -- Take the chatbot code. Add streaming. Change the system prompt to a creative personality (pirate, fitness coach, Bollywood villain).
2. **Pydantic Validation** -- Pick a domain (Movie, Recipe, Book). Define a Pydantic model. Ask AI to generate data. Validate it. See if it ever fails.
3. **(Optional)** Create a virtual environment on your laptop. Install `google-generativeai` and `pydantic`. Run a simple script.

Prep for Session 6

- Google Colab is sufficient (nothing new to install)
- Topics: Transformer architecture deep dive, self-attention mechanics, tokenizers, embeddings, HuggingFace Transformers library

Session Whiteboard

Session 5 - All Diagrams



Notebook Code (Full Reference)

All code from the session. Uses Ollama (local, free, no API key).

Setup: `pip install ollama pydantic` and run `ollama pull llama3.2`

```
import ollama

# Ollama runs locally -- no API key needed!
# Make sure Ollama is running: open terminal, type: ollama serve
# Then pull a model: ollama pull llama3.2
print("Ollama library loaded!")
```

```
# Quick test: is Ollama running?
try:
    models = ollama.list()
    print("Ollama is running!")
    print("Available models:", [m.model for m in models.models])
except Exception as e:
    print(f"Ollama not running! Start it with: ollama serve")
    print(f"Error: {e}")
```

```
def ask_ai(prompt):
    response = ollama.chat(
        model="llama3.2",
        messages=[
            {"role": "system", "content": "You are a helpful assistant. Keep responses to 2-3 sentences."},
            {"role": "user", "content": prompt}
        ]
    )
    return response.message.content
```

```
answer = ask_ai("What is Python in one sentence?")
print(answer)
```

```
# No memory -- model doesn't know what 'there' means
print(ask_ai("What is the capital of France?"))
```

```
print(ask_ai("What language do they speak there?"))
```

```

def chat():
    messages = [
        {"role": "system", "content": "You are a friendly AI assistant. Keep responses to 2-3 sentences."}
    ]

    print("Chatbot ready! Type 'quit' to exit.\n")

    while True:
        user_input = input("You: ")

        if user_input.lower() == "quit":
            print("Goodbye!")
            break

        messages.append({"role": "user", "content": user_input})

        response = ollama.chat(
            model="llama3.2",
            messages=messages
        )

        assistant_message = response.message.content
        messages.append({"role": "assistant", "content": assistant_message})

        print(f"AI: {assistant_message}\n")

```

```
chat()
```

```

def ask_ai_stream(prompt):
    stream = ollama.chat(
        model="llama3.2",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": prompt}
        ],
        stream=True
    )

    full_response = ""
    for chunk in stream:
        word = chunk.message.content
        print(word, end="", flush=True)
        full_response += word

    print()
    return full_response

```

```
# Without streaming
print("--- Without streaming ---")
answer = ask_ai("Tell me a fun fact about space")
print(answer)
```

```
# With streaming -- watch words appear one by one
print("--- With streaming ---")
answer = ask_ai_stream("Tell me a fun fact about the ocean")
```

```
from pydantic import BaseModel
```

```
class Person(BaseModel):
    name: str
    age: int
    city: str
```

```
# Valid data
person1 = Person(name="Priya", age=24, city="Mumbai")
print(person1)
print(f"Name: {person1.name}")
```

```
# Invalid data -- REJECTED
try:
    bad_person = Person(name="Rahul", age="twenty-five", city="Delhi")
except Exception as e:
    print(f"REJECTED: {e}")
```

```
# Missing field -- REJECTED
try:
    incomplete = Person(name="Arun", age=30)
except Exception as e:
    print(f"REJECTED: {e}")
```

```
import json

# Ask AI for structured data
response = ask_ai(
    "Give me a fictional person with name, age, and city. "
    "Respond ONLY in JSON format like: {\n\"name\": \"...\", \n\"age\": ..., \n\"city\": \"...\"}"
)
print("Raw AI response:", response)
```



```
# Validate AI response
try:
    data = json.loads(response)
    person = Person(**data)
    print(f"VALID! Name: {person.name}, Age: {person.age}, City: {person.city}")
except json.JSONDecodeError:
    print("AI didn't return valid JSON!")
except Exception as e:
    print(f"Validation failed: {e}")
```

```
import time

questions = [
    "What is gravity in one sentence?",
    "What is photosynthesis in one sentence?",
    "What is democracy in one sentence?"
]
```

```
# SEQUENTIAL: one by one
start = time.time()
answers = []
for q in questions:
    answers.append(ask_ai(q))
end = time.time()

print(f"Sequential: {end - start:.1f} seconds")
for i, a in enumerate(answers):
    print(f"  Q{i+1}: {a}")
```

```
import asyncio

# PARALLEL: all at once
async def ask_ai_async(prompt):
    return await asyncio.to_thread(ask_ai, prompt)

async def ask_all():
    tasks = [ask_ai_async(q) for q in questions]
    return await asyncio.gather(*tasks)

start = time.time()
answers = await ask_all()
end = time.time()

print(f"Parallel: {end - start:.1f} seconds")
for i, a in enumerate(answers):
    print(f"  Q{i+1}: {a}")
```

```
# Write a text file
with open("hello.txt", "w") as f:
    f.write("This is my first file from Python!")
print("Written!")
```

```
# Read it back
with open("hello.txt", "r") as f:
    content = f.read()
print(content)
```

```
import json

# Write a config as JSON
config = {
    "model": "llama3.2",
    "system_prompt": "You are a helpful coding tutor.",
    "temperature": 0.7
}
with open("config.json", "w") as f:
    json.dump(config, f, indent=2)
print("Config saved!")
```

```
# Read JSON back
with open("config.json", "r") as f:
    cfg = json.load(f)
print(cfg["model"])
print(cfg["system_prompt"])
```

```
# Use config to control AI
response = ollama.chat(
    model=cfg["model"],
    messages=[
        {"role": "system", "content": cfg["system_prompt"]},
        {"role": "user", "content": "How do I write a for loop?"}
    ]
)
print(response.message.content)
```

```
print("""
VIRTUAL ENVIRONMENT COMMANDS (for your terminal):

1. Create:    python -m venv myproject_env
2. Activate:  source myproject_env/bin/activate  (Mac/Linux)
               myproject_env\\Scripts\\activate  (Windows)
3. Install:   pip install ollama pydantic
4. Check:     pip freeze
5. Exit:      deactivate

To run Ollama:
  1. Download from https://ollama.com
  2. Terminal: ollama serve
  3. Terminal: ollama pull llama3.2
""")
```